

VulnBot: Autonomous Penetration Testing for A Multi-Agent Collaborative Framework

He Kong^{1,2}, Die Hu^{1,2}, Jingguo Ge^{1,2}, Liangxiong Li¹, Tong Li¹, and Bingzhen Wu¹

¹*State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering,
Chinese Academy of Sciences*

²*School of Cyber Security, University of Chinese Academy of Sciences*

Abstract

Penetration testing is a vital practice for identifying and mitigating vulnerabilities in cybersecurity systems, but its manual execution is labor-intensive and time-consuming. Existing large language model (LLM)-assisted or automated penetration testing approaches often suffer from inefficiencies, such as a lack of contextual understanding and excessive, unstructured data generation. This paper presents VulnBot, an automated penetration testing framework that leverages LLMs to simulate the collaborative workflow of human penetration testing teams through a multi-agent system. To address the inefficiencies and reliance on manual intervention in traditional penetration testing methods, VulnBot decomposes complex tasks into three specialized phases: reconnaissance, scanning, and exploitation. These phases are guided by a penetration task graph (PTG) to ensure logical task execution. Key design features include role specialization, penetration path planning, inter-agent communication, and generative penetration behavior. Experimental results demonstrate that VulnBot outperforms baseline models such as GPT-4 and Llama3 in automated penetration testing tasks, particularly showcasing its potential in fully autonomous testing on real-world machines.

1 Introduction

Penetration testing is a critical methodology for proactively identifying network vulnerabilities and mitigating potential cyberattacks [5, 21]. It enables the timely detection of weaknesses in target systems, facilitating targeted remediation and reinforcement efforts. According to market forecasts, the penetration testing market is projected to grow significantly, expanding from US \$1.92 billion in 2023 to US \$6.98 billion by 2032 [11]. Despite its importance, traditional penetration testing remains a labor-intensive and time-consuming process, requiring highly skilled professionals to execute complex workflows manually. As network threats continue to grow in both complexity and scale, there is an urgent need for more efficient, scalable, and automated penetration testing

methodologies to reduce manual effort and enhance testing efficiency [9].

Recent advancements in large language models (LLMs) and multi-agent systems have opened new avenues for automating penetration testing [2, 10, 18, 30, 35, 47, 50]. Consequently, researchers have proposed various approaches to leverage LLMs for automated penetration testing. For instance, Deng et al. introduced PentestGPT, a pioneering effort to utilize LLMs for automating penetration testing [14]. PentestGPT addresses the issue of context loss in LLMs during penetration testing through three interconnected modules: a reasoning module, a generation module, and a parsing module. However, PentestGPT heavily relies on human intervention and cannot assess the extent of such involvement, resulting in limited agent autonomy. In contrast, AutoAttacker, a novel method, focuses on automating the post-penetration phase of simulated network attacks (i.e., "keyboard-operated" attacks) [56]. By employing a modular design, AutoAttacker leverages the planning, summarization, and code generation capabilities of LLMs, combined with tools like Metasploit, to demonstrate the efficacy of LLMs in isolated security tasks. Nevertheless, AutoAttacker primarily targets specific tasks rather than real-world environments. While existing studies have explored the use of LLMs for automated penetration testing, they are often limited in scope, focusing on specific tasks or relying on detailed vulnerability descriptions, which are difficult to apply in real-world scenarios. Additionally, these methods predominantly depend on the GPT-4 model [2], making it challenging to execute complex tasks using open-source models.

In this paper, we present VulnBot, an autonomous, multi-agent penetration testing framework based on LLMs, designed to emulate the collaborative workflows of human penetration testing teams. By integrating specialized modules that focus on different phases of penetration testing, VulnBot aims to automate and streamline the process of identifying vulnerabilities in target systems. The framework incorporates distinct roles, such as reconnaissance, scanning, and exploitation, along with a Penetration Task Graph (PTG)-based approach

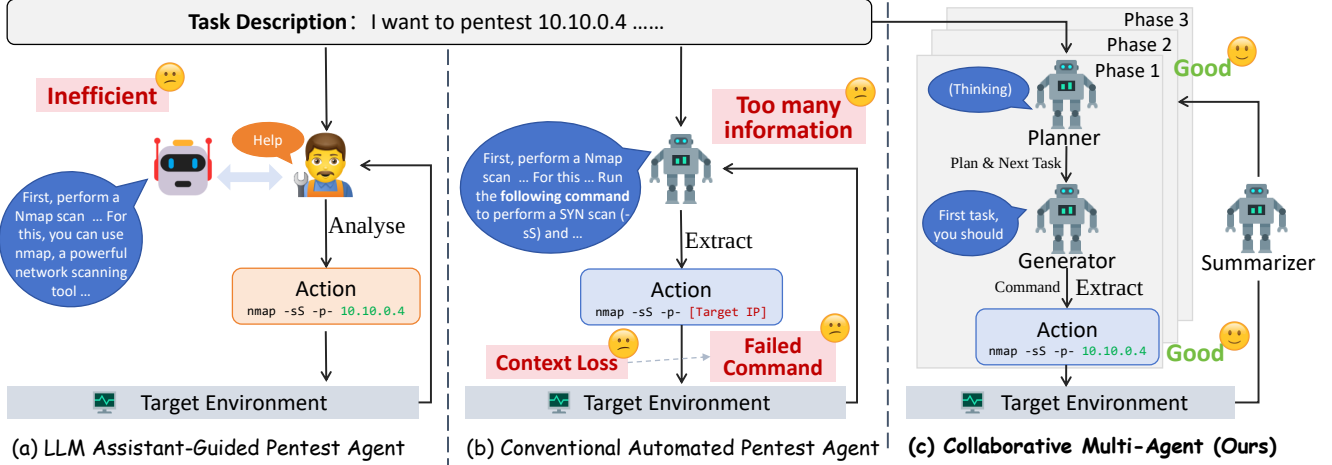


Figure 1: The workflow comparison of three approaches to automated penetration testing: (a) LLM Assistant-Guided Pentest Agent, which requires assistance due to inefficiency; (b) Conventional Automated Pentest Agent, which struggles with information overload and context loss; and (c) Collaborative Multi-Agent system, which employs a phased and modular approach, enhancing the overall efficiency and autonomy of the penetration testing process through multi-agent coordination.

to penetration path planning, inter-agent communication, and generative penetration behavior. These components work together to simulate a robust and comprehensive penetration testing workflow.

At the core of VulnBot’s design is its ability to model the penetration testing process as a series of interdependent tasks, each contributing to the overarching goal of identifying and exploiting vulnerabilities in the target system. The PTG organizes these tasks, ensuring that each step is executed in the correct sequence and context. VulnBot enhances inter-agent communication through a Summarizer module, which acts as a bridge between different phases of penetration testing. By summarizing key task outcomes and transmitting them to subsequent roles, the Summarizer ensures that critical information is preserved and prioritized across stages. This targeted communication minimizes redundancy, ensures clarity, and optimizes the flow of information across agents, thereby maintaining the integrity and continuity of the penetration testing process. Furthermore, VulnBot’s Generator and Executor modules translate these tasks into tool-specific commands, simulate human-like interactions with the target system, and autonomously execute them, significantly reducing the need for continuous human oversight. This paper introduces three operational modes: automatic, semi-automatic, and human-involved. The experimental evaluation focuses on the automatic mode, as human involvement introduces subjectivity and variability that are difficult to quantify.

We evaluated VulnBot across two distinct benchmarks to assess its performance and real-world applicability. On the AUTOPENBENCH, VulnBot significantly outperformed baseline models, including GPT-4o, Llama3.3-70B, and Llama3.1-

405B. Specifically, VulnBot-Llama3.1-405B achieved a completion rate of 30.3%, compared to 9.09% for Llama3.1-405B and 21.21% for GPT-4o. Additionally, VulnBot demonstrated superior performance in the early stages of the test. By delaying the automation of penetration testing to later stages, VulnBot ensures that critical subtasks are executed with greater precision, thereby increasing the likelihood of completing the testing process. Ablation studies further confirmed the effectiveness of the various components within the framework. On real-world machines using the AI-Pentest-Benchmark, VulnBot—when paired with Llama3.1-405B and DeepSeek-v3—surpassed other baseline models. When integrated with Retrieval Augmented Generation (RAG), VulnBot’s performance further improved. In real-world machines, VulnBot with RAG autonomously completed tasks end-to-end, a feat that GPT-4o and Llama3.1-405B, which relied on human intervention, could not achieve. These results highlight VulnBot’s potential for fully autonomous penetration testing.

The contributions of this work are as follows:

- We introduce VulnBot, an autonomous penetration testing framework that leverages the capabilities of LLMs and multi-agent systems to automate complex penetration testing workflows. Inspired by the collaborative dynamics of human penetration testing teams, VulnBot employs a tri-phase design—reconnaissance, scanning, and exploitation. This design ensures that agents focus on specific tasks, minimizing information loss and enhancing efficiency.
- We propose a task-driven mechanism based on a PTG, which models tasks and their dependencies as a directed

acyclic graph. The PTG ensures that tasks are executed in a logical and conflict-free order, providing a structured framework for tracking task progress and outcomes. This mechanism, combined with a Check and Reflection Mechanism, enables continuous improvement and adaptation of the plan based on task status, achieving effective error handling and feedback.

- By utilizing open-source models such as Llama3.3-70B, Llama3.1-405B, and DeepSeek-V3, we demonstrate the feasibility and effectiveness of leveraging open-source LLMs for automated penetration testing. Our experimental results show that VulnBot outperforms baseline models such as GPT-4 and Llama3, achieving a 69.05% subtask completion rate and a 30.3% overall completion rate on the AUTOPENBENCH. Additionally, VulnBot achieved the best performance on six real-world machines in the AI-Pentest-Benchmark. Through the integration of RAG, VulnBot successfully realized complete end-to-end penetration of real-world machines.

2 Background & Motivation

2.1 Background

Penetration testing, also referred to as ethical hacking, is a method employed to evaluate the security of computer systems, networks, or applications by simulating potential malicious attacks. The primary objective is to identify and remediate potential vulnerabilities before they can be exploited by real attackers [1, 11]. According to the OWASP Testing Guide [45], penetration testing typically consists of five key phases: reconnaissance, scanning, vulnerability exploitation, maintaining access, and reporting [8]. The duration of these phases varies depending on the scope of the test. On average, the entire process takes approximately 10 days, with the reconnaissance phase being the most time-consuming, often lasting between 4 to 6 days [7, 53]. The cost of penetration testing is also influenced by the type and scope of the assessment. For example, a basic website scan typically costs between US \$349 and US \$1499, while more comprehensive assessments, such as SaaS or web application scanning, can range from US \$700 to US \$5999 [46, 49].

2.2 Motivation

Traditional penetration testing is both time-intensive and costly, highlighting the need for more efficient, automated solutions. Current approaches leveraging LLM-assisted or automated agents for penetration testing face notable inefficiencies. As illustrated in Figure 1, the LLM Assistant-Guided Pentest Agent (a) lacks autonomy and requires frequent user intervention to clarify tasks, resulting in inefficiency. Similarly, the Conventional Automated Penetration Testing Agent

(b) generates an excessive amount of unstructured data but fails to provide actionable insights or clear next steps, leading to context loss and command failures. The Collaborative Multi-Agent System (c) addresses these limitations by leveraging specialized agents for reconnaissance, scanning, and exploitation. This system employs a modular and phased approach, effectively managing tasks through coordinated planning, generation, and summarization, thereby enhancing the overall efficiency and autonomy of the penetration testing process.

2.2.1 Task Definition

In this paper, we define autonomous penetration testing as encompassing two types of tasks: those requiring human intervention, where penetration testers provide guidance, and those conducted entirely without human intervention. This work specifically focuses on the latter—tasks performed autonomously, without the need for human oversight. Due to time and cost constraints, we leverage open-source models to minimize expenses.

2.2.2 Exploratory Study

Before investigating methods for automated penetration testing, we conducted an empirical study to address three research questions that explore the challenges of using open-source LLMs in this domain:

RQ1: To what extent can open-source LLMs perform penetration testing tasks?

To address RQ1, we reviewed the existing literature and evaluated the performance of open-source LLMs in penetration testing contexts. Isozaki et al. conducted an analysis of open-source LLMs, introducing the AI-Pentest-Benchmark, which comprises 13 real machines from Vulnhub [33, 51]. They tested two prominent models, GPT-4o and Llama3.1-405B, using the PentestGPT tool. The study revealed that Llama3.1-405B outperformed GPT-4o in reconnaissance and exploitation tasks for machines of easy and medium difficulty. However, both models encountered challenges in privilege escalation and tasks involving high-difficulty machines.

RQ2: What are the reasons for the failure of penetration testing using open-source LLMs?

RQ3: How do open-source LLMs perform in the different phases of penetration testing?

To answer RQ2 and RQ3, we utilized the AUTOPENBENCH benchmark [22], which includes 33 tasks designed to simulate real-world penetration testing scenarios. These tasks are categorized into two difficulty levels: in-vitro tasks (basic network security scenarios) and real-world tasks (based on publicly disclosed CVEs). We conducted further analysis of the in-vitro tasks using the 128k-context Llama3.3-70B and Llama3.1-405B models, with each test executed five times, to

Model	Phase	Failure Count	Cause of Failure					
			Session Context Loss	False Output Interpretation	Failed Tool	Deadlock Operation	Failed Command Param	Other
Llama3.3-70B	Reconnaissance	28	18 (64.29%)	3 (10.71%)	2 (7.14%)	0 (0.00%)	4 (14.29%)	1 (3.57%)
	Scanning	27	5 (18.52%)	2 (7.41%)	8 (29.63%)	3 (11.11%)	7 (25.93%)	2 (7.41%)
	Exploitation	45	16 (35.56%)	4 (8.89%)	13 (28.89%)	2 (4.44%)	8 (17.78%)	2 (4.44%)
Llama3.1-405B	Reconnaissance	43	28 (65.12%)	4 (9.30%)	1 (2.33%)	5 (11.63%)	3 (6.98%)	2 (4.65%)
	Scanning	27	4 (14.81%)	3 (11.11%)	9 (33.33%)	0 (0.00%)	11 (40.74%)	0 (0.00%)
	Exploitation	33	15 (45.45%)	2 (6.06%)	7 (21.21%)	1 (3.03%)	6 (18.18%)	1 (3.03%)
Total		203	86 (42.36%)	18 (8.87%)	40 (19.70%)	11 (5.42%)	39 (19.21%)	8 (3.94%)

Table 1: Failure counts and causes for open-source LLMs in different phases

gain a deeper understanding of the limitations of open-source models in penetration testing tasks.

To investigate the causes of failure in penetration testing experiments, we conducted a detailed analysis and classification of the results from 220 experiments, as summarized in Table 1. The primary cause of failure across both the reconnaissance and exploitation stages was the loss of session context. However, the specific causes of failure varied between stages. During the reconnaissance phase, the models frequently struggled to understand the initial description provided by the user, such as failing to execute commands like `nmap -p 10.10.1.x` for a comprehensive port scan. In the exploitation phase, the models often forget previously scanned target machines or relevant information obtained during earlier stages. This context loss is primarily attributed to two factors: the limited size of the context window and the token constraints inherent to LLMs. When the critical data from a complex task exceeds the available context, important details may be truncated, leading to the loss of vital information. Moreover, if the execution results are excessively lengthy, the context can become overloaded, further diminishing the model’s effectiveness. Overall, Table 1 emphasizes the need for targeted improvements in session management, tool reliability, and command parameter accuracy to enhance the robustness of open-source LLMs in practical applications.

2.3 Challenge

Takeaway#1: LLM Context Length. A significant limitation of LLMs is their fixed context length, which impedes their ability to maintain a coherent understanding of the entire penetration testing process. As the model progresses through the various stages of the test, it often loses track of earlier discoveries, leading to a failure to leverage prior insights. This context loss causes the model to forget critical steps or findings, thereby hindering task completion and adversely affecting overall performance.

Takeaway#2: Penetration Command Generation. LLMs frequently encounter difficulties in generating accurate penetration testing commands. They may produce incorrect tool usage or fabricate non-existent parameters. Automated pene-

tration testing requires precise translation of natural language instructions into executable commands. However, the inability of current LLMs to reliably perform this translation introduces significant errors and inefficiencies, undermining the accuracy and reliability of the testing process.

Takeaway#3: Lack of Effective Error-Handling Mechanism. Current LLM-based systems lack an effective error-handling mechanism to manage command execution failures or anomalies. When an error occurs, the model typically cannot autonomously diagnose the issue or take corrective actions. Consequently, manual intervention is often required to resolve problems and resume testing, which diminishes the overall automation and efficiency of the system.

Takeaway#4: Dynamic Reasoning Across Testing Phases. Penetration testing involves multiple, interdependent phases—reconnaissance, scanning, exploitation, and post-exploitation—each of which builds on the information gathered in previous stages. For effective automation, it is not enough for a system to perform well in isolated phases; it must also integrate findings dynamically to guide subsequent actions. For example, insights gained during scanning must inform exploitation strategies. Current systems struggle to maintain this dynamic flow, often requiring human oversight to link findings across phases. This limitation results in fragmented analyses, where critical connections between discoveries are missed. The inability to synthesize information across multiple stages becomes especially problematic in complex scenarios.

3 Design

In this section, we present the design of VulnBot, an autonomous penetration testing framework for LLM-based multi-agent systems. We begin by providing an overview of the overall architecture of VulnBot in Section 3.1. Subsequently, we elaborate on the four key design aspects of VulnBot: (1) specialization of roles (Section 3.2), (2) penetration path planning, which incorporates the Planner and Memory Retriever modules (Section 3.3), (3) inter-agent communication, facilitated by the Summarizer module (Section 3.4), and (4) generative penetration behavior and interaction, en-

abled by the Generator and Executor modules (Section 3.5).

3.1 Overview

The overall architecture of VulnBot is illustrated in Figure 2. VulnBot is an autonomous penetration testing framework designed to emulate the collaborative and strategic workflows of human penetration testing teams. The system is built around five core modules—Planner, Memory Retriever, Generator, Executor, and Summarizer—which collectively automate the three primary phases of penetration testing: Reconnaissance, Scanning, and Exploitation. This design addresses the complexities of automating penetration testing tasks while ensuring adaptability to unforeseen challenges, thereby maintaining robustness across diverse testing scenarios.

3.2 Specialization of Roles

Drawing from Takeaways 1 and 4, we employ a specialization of roles mechanism. Clear role specialization is a cornerstone of effective problem-solving in complex systems. By decomposing intricate tasks into smaller, well-defined subtasks, specialized roles enable agents to focus on specific objectives, leveraging their unique expertise to contribute to the overall goal. In the context of penetration testing, this approach is particularly critical, as the process involves multiple interdependent phases, each requiring distinct skills and tools.

Our design encountered a significant challenge stemming from the context length limitations of LLMs. When executing the five-phase process, critical information from earlier phases is often lost or diluted as the process progresses. This occurs because each phase must retain and reference information from all preceding phases, not just the immediate prior one. To address this limitation, we restructured the penetration testing process into three specialized phases: reconnaissance, scanning, and exploitation. This streamlined approach ensures that each phase maintains a clear focus while minimizing information loss across transitions. We provide the agent with task instructions in the form of text, including the task description, a role-playing jailbreak method [13, 38, 56] to bypass LLM usage policies, and additional preliminary information about the agent.

Reconnaissance The reconnaissance phase serves as the foundation of the penetration testing process, aimed at gathering comprehensive information about the target system. In this phase, agents are tasked with performing a full scan of the target to identify all open ports and services. To achieve this, we equip reconnaissance agents with tools such as Nmap [44] and Dirb [17], which are widely used for network discovery. By systematically collecting and organizing this data, the reconnaissance phase provides the necessary context for the subsequent scanning phase.

Scanning Building on the data gathered during reconnaissance, the scanning phase focuses on identifying vulnerabili-

ties and misconfigurations within the target system. Agents in this phase utilize specialized tools such as Nikto [43] (for web server vulnerability scanning) and WPScan [54] (for identifying issues with WordPress sites) to detect potential weaknesses. The scanning phase is critical for narrowing down the attack surface and prioritizing vulnerabilities that are most likely to be exploitable. By maintaining a clear separation between reconnaissance and scanning, we ensure that agents can focus on their specific tasks without being overwhelmed by extraneous information.

Exploitation The exploitation phase marks the culmination of the penetration testing process, where vulnerabilities discovered during reconnaissance and vulnerability scanning are exploited to gain access to the target system and escalate privileges. In this phase, agents are equipped with tools such as Metasploit [39] (for developing and executing exploit code) and Hydra [32] (for brute-forcing credentials).

This design ensures that each phase builds upon the previous one, enabling a seamless and effective workflow that adapts to the complexities of real-world systems.

3.3 Penetration Path Planning

Penetration path planning is a critical component of VulnBot, which incorporates the Planner and Memory Retriever modules. The Planner module is responsible for generating and maintaining the penetration testing plan. It operates through two distinct sessions: the Plan Session and the Task Session, each serving a specific purpose in the planning and execution process.

Plan Session: The Planner initially generates an action plan in a JSON-compliant structure, tailored to the user’s requirements and the characteristics of the target system. This plan is decomposed into structured task lists, each defined by unique identifiers, dependencies, instructions, and action types. As shown in Figure 3. The primary objective is to construct a Penetration Testing Task Graph (PTG), which outlines the logical sequence of tasks to be executed. Subsequently, the plan is dynamically updated based on the results of task execution, incorporating feedback from both successful and failed tasks.

This session is governed by two key mechanisms: the **Task-driven Mechanism** (Section 3.3.1), which organizes tasks into a directed acyclic graph, and the **Check and Reflection Mechanism** (Section 3.3.2), which ensures continuous improvement and adaptation of the plan through iterative feedback from task execution result.

Task Session: This session focuses on generating specific task details for each instruction, which are then fed into the Generator for execution. Additionally, it is responsible for checking task execution results success.

To mitigate the hallucination problem often associated with LLMs, we employ a third-party retrieval-augmented generation framework, Langchain-Chatchat [37]. The Memory Re-